Git And Git-hub Challenges -1

Notes: Create a Jira ticket for this assignment, work on the tasks listed below, and submit the challenge completion document by EOD with clear steps and required evidence (outputs/screenshots: Git Challenges

Resolve Merge Conflicts

Create a merge conflict intentionally (two users editing the same line).

Resolve the conflict and push the changes.

Recover Deleted Branch

Delete a local branch and then recover it using the reflog.

Undo Wrong Push

Push a wrong commit to GitHub, then undo it without losing history.

Amend a Commit

Make a commit, then add a missing file to it using git commit --amend.

Cherry-pick a Commit

Take a specific commit from one branch and apply it to another branch.

Interactive Rebase

Reorder and squash multiple commits into a single clean commit.

Tagging & Release

Create a version tag (v1.0), push it to GitHub, then delete and restore it.

Clone with Sparse Checkout

Clone only a subdirectory of a repo using sparse checkout.

Reset vs Revert Challenge

Demonstrate the difference between git reset --hard and git revert in a repo.

Detached HEAD Challenge

Checkout a specific commit (detached HEAD state) and create a new branch from it.

Git Hooks Challenge

Configure a pre-commit hook to reject commits without a message format (e.g., must start with JIRA-XXX).

Squash Merge vs Rebase Merge

Show the difference between squash merge and rebase merge with evidence.

Fork & Pull Request Workflow

Fork a repo, make a change, and submit a pull request to the original repo.

Recover Lost Commit

Commit something, reset hard, and then recover it using git reflog.

1) **Resolve Merge Conflicts**

   Create a merge conflict intentionally (two users editing the same line).

   Resolve the conflict and push the changes.

1. Local Repository Setup & Initial Commit
mkdir my-project – Creates a new project folder.
cd my-project – Navigates into the project directory.
git init – Initializes a local Git repository.
git add <file> – Stages specific changes (e.g., git add note.txt).
git commit -m "<message>" – Commits staged changes with a message.
2. Branching & Feature Development
git checkout -b <branch-name> – Creates and switches to a new branch (e.g., git checkout -b feature).
git commit -am "<message>" – Automatically stages and commits all modified tracked files.
3. Merging & Conflict Resolution
git checkout <branch> – Switches between branches (e.g., git checkout -m master).
git merge <branch> – Merges another branch into the current branch. (Triggers a conflict if the same file was modified differently).
vi <file> – Opens the file in an editor to manually resolve conflicts.
git add <file> – Marks the conflict as resolved by staging the file.

git commit -m "<message>" – Concludes and finalizes the merge.

4. Remote Synchronization & Pushing

git push -u origin <branch> – Pushes local commits to the remote repository and sets the upstream track.

git pull origin <branch> – Fetches and merges remote changes into the local branch to resolve divergence.

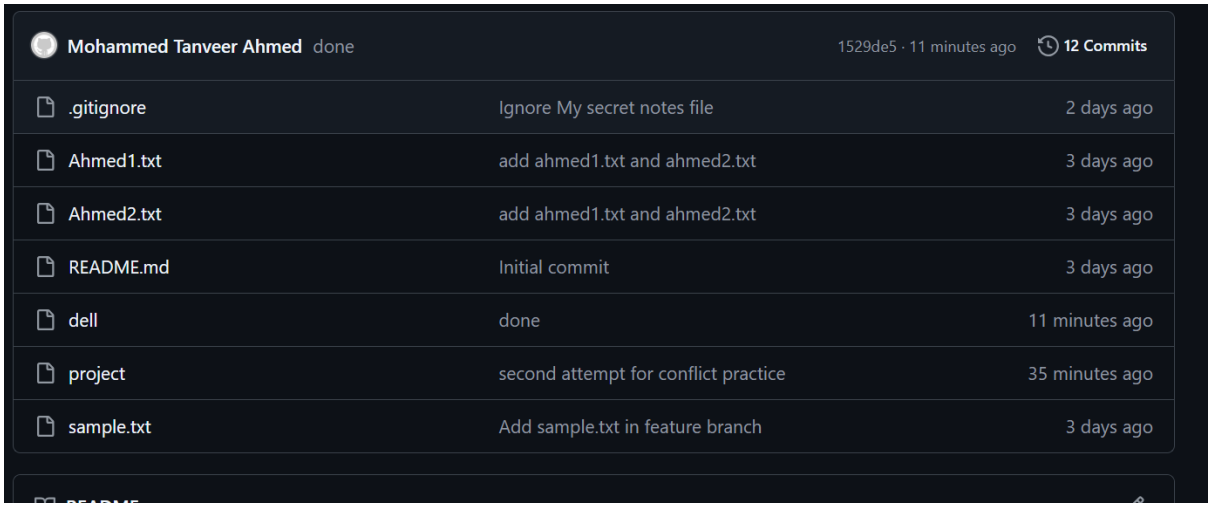
```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (develo
pment)
$ git commit -m "added develp file"
[development d8150fa] added develp file
 1 file changed, 2 insertions(+)
 create mode 100644 dell

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (develo
pment)
$ git merge hotfix
Auto-merging dell
CONFLICT (add/add): Merge conflict in dell
Automatic merge failed; fix conflicts and then commit the result.

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (develo
```

```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (develo
pment)
$ git push -u origin development
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), 799 bytes | 399.00 KiB/s, done.
Total 9 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 1 local object.
remote:
remote: Create a pull request for 'development' on GitHub by visiting:
remote:      https://github.com/mdtanveer801/Git_practice2/pull/new/de
velopment
remote:
To github.com:mdtanveer801/Git_practice2.git
 * [new branch]      development -> development
branch 'development' set up to track 'origin/development'.

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (develo
pment)
$
```



Recover Deleted Branch

- Delete a local branch and then recover it using the reflog.

```
pment)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (main)
$ git branch -D development
Deleted branch development (was c4b2ac3).

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (main)
$ git reflog
c4b2ac3 (HEAD -> main, origin/main, origin/HEAD) HEAD@{0}: checkout: m
oving from development to main
c4b2ac3 (HEAD -> main, origin/main, origin/HEAD) HEAD@{1}: checkout: m
oving from main to development
c4b2ac3 (HEAD -> main, origin/main, origin/HEAD) HEAD@{2}: checkout: m
oving from development to main
1529de5 (origin/development) HEAD@{3}: commit (merge): done
d8150fa HEAD@{4}: commit: added develp file
c5c7796 HEAD@{5}: checkout: moving from hotfix to development
2303866 (hotfix) HEAD@{6}: commit: added a file
c5c7796 HEAD@{7}: checkout: moving from main to hotfix
c4b2ac3 (HEAD -> main, origin/main, origin/HEAD) HEAD@{8}: commit: sec
ond attempt for 2
4030faa (feature) HEAD@{9}: merge feature: Fast-forward
c5c7796 HEAD@{10}: checkout: moving from feature to main
4030faa (feature) HEAD@{11}: commit: second attempt for conflict2
c5c7796 HEAD@{12}: checkout: moving from main to feature
c5c7796 HEAD@{13}: commit: second attempt for conflict practice
f872dde HEAD@{14}: clone: from github.com:mdtanveer801/Git_practice2.g
it

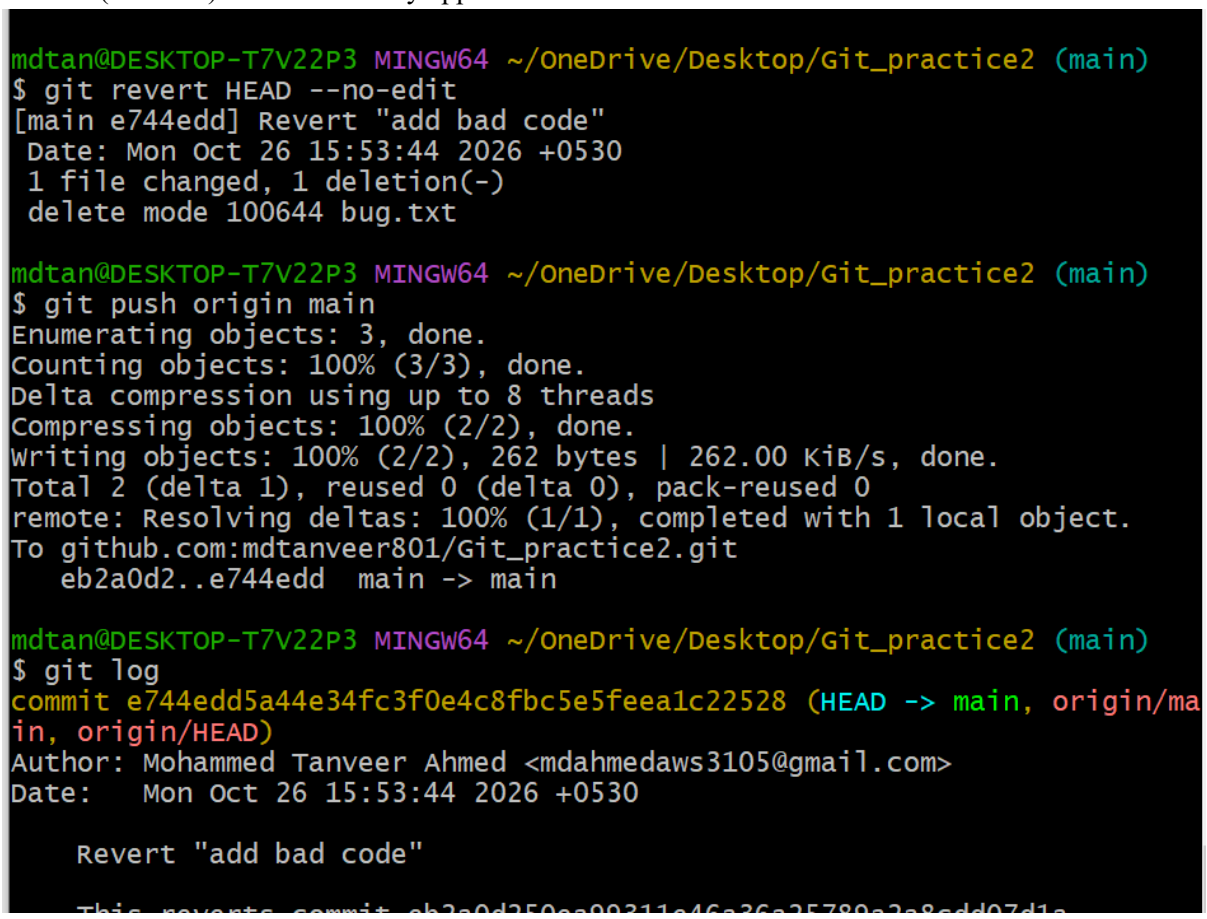
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (main)
$ git checkout -b development 1529de5
Switched to a new branch 'development'

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (develo
pment)
$
```

Advanced Branching & History Recovery (Reflog)
Switching Back to Main:
git checkout main – Switches the active working context back to the main branch.
Deleting a Branch:
git branch -D development – Force-deletes a local branch (in this case, development).
Viewing Repository History:
git reflog – Displays a log of all recent movements of the HEAD pointer (checkouts, commits, merges, and resets), which helps track past states and recover lost work. Recreating a Branch from a Specific Commit:
git checkout -b development 1529de5 – Creates and switches to a new branch named development starting from a specific historical commit hash (1529de5) found via reflog.

3)**Undo Wrong Push**
Push a wrong commit to GitHub, then undo it without losing history.
Reverting Changes & Syncing with Remote

Reverting a Commit: git revert HEAD --no-edit – Safely undoes the changes from the latest commit (HEAD) by creating a new commit that inverses those changes, skipping the text editor prompt to edit the commit message (in this case, deleting bug.txt).
Pushing Revert Changes: git push origin main – Uploads the new local revert commit to the remote repository on GitHub (origin/main).
Verifying Commit History: git log – Displays the repository's commit history to verify that the revert commit (e744edd) was successfully applied and recorded on the active main branch.



4) Amend a Commit

Make a commit, then add a missing file to it using git commit --amend.

File Creation, Staging, and Amending Commits
Creating and Staging Files:
echo "<text>" > <file> – Creates a new file and writes text into it (e.g., echo "welcome to laptop" > ball.txt).
git status – Displays the repository status, highlighting untracked files and whether the local branch is ahead of the remote. git add <file> – Stages a specific file to prepare it for the next commit (e.g., git add ball.txt and git add cat.txt).
Committing and Amending:
git commit -m "<message>" – Records staged changes into the repository history with a message (e.g., git commit -m "my new code added").
git commit --amend – Modifies the most recent commit by combining newly staged changes (such as adding cat.txt right after) into the previous commit instead of creating a separate one.

```
$ echo "welcome to laptop" >ball.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ball.txt
        main application code

nothing added to commit but untracked files present (use "git add" to tr
ack)

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (main)
$ git add ball.txt
warning: in the working copy of 'ball.txt', LF will be replaced by CRLF
the next time Git touches it

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (main)
$ git add ball.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (main)
$ git commit -m "my new code added"
[main 97f958c] my new code added
 1 file changed, 1 insertion(+)
 create mode 100644 ball.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (main)
$ echo "welcome to oldest laptop" >cat.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (main)
$ git add cat.txt
warning: in the working copy of 'cat.txt', LF will be replaced by CRLF t
he next time Git touches it

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (main)
$ git commit --amend
[main a54b935] two files are combines successfully in one commit.
 Date: Mon Oct 26 16:11:08 2026 +0530
 2 files changed, 2 insertions(+)
 create mode 100644 ball.txt
 create mode 100644 cat.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (main)
$
```

5) Cherry-pick a Commit

Take a specific commit from one branch and apply it to another branch.

Branch Management, Commit Logs, and Cherry-Picking

Listing and Switching Branches:

git branch --list – Displays a list of all local branches in the repository.

git checkout <branch> – Switches your active working environment to a specific branch (e.g., git checkout feature).

Viewing Branch History: git log <branch> --oneline – Displays a compact, single-line commit history for a specified branch to easily inspect past changes and commit hashes.

Cherry-Picking Commits: git cherry-pick <commit-hash> – Pulls a specific commit from anywhere in your history and applies its changes directly onto your current active branch without merging the whole branch (e.g., git cherry-pick eb2a0d2 to bring over bug.txt). Verifying Directory Contents: ls – Lists the files and folders in the current directory to verify that the cherry-picked changes (such as bug.txt) were successfully applied.

```
$ git branch --list
  development
  feature
  hotfix
* main
  release

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (main)
$ git checkout feature
Switched to branch 'feature'

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$ git log main --online
fatal: unrecognized argument: --online

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$ git log main --oneline
a54b935 (release, main) two files are combines successfully in one commit.
9587a7d my forget code added
07bd91c my first code added
e744edd (origin/main, origin/HEAD) Revert "add bad code"
eb2a0d2 add bad code
c4b2ac3 second attempt for 2
4030faa (HEAD -> feature) second attempt for conflict2
c5c7796 second attempt for conflict practice
f872dde Revert "added a new file1 in main branch"
6d721ed Ignore My secret notes file
bc9e098 add passwords.txt to gitignore
b557cec (origin/hotfix) added a new file1 in main branch
dd3c2e4 Merge pull request #1 from mdtanveer801/feature-branch
f50c0b6 (origin/feature-branch) Add sample.txt in feature branch
61eddc5 add ahmed1.txt and ahmed2.txt
3edf1a1 Initial commit

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$ git cherry-pick eb2a0d2
[feature fa5e523] add bad code
 Date: Mon Oct 26 15:52:32 2026 +0530
 1 file changed, 1 insertion(+)
 create mode 100644 bug.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$ ls
 Ahmed1.txt   README.md  'main application code'   project
 Ahmed2.txt   bug.txt     myproject/               sample.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$
```

6) Interactive Rebase

Reorder and squash multiple commits into a single clean commit.

Interactive Rebase (Squashing Commits)

Creating Sequential Commits:

echo "<text>" > <file> / echo "<text>" >> <file> – Creates or appends code/text updates into a file across multiple steps (e.g., creating feature.txt with steps 1, 2, and 3).

git commit -am "<message>" – Stages and commits each sequential update with distinct messages ("doing", "fixing", "final touchups").

Starting an Interactive Rebase: git rebase -i HEAD~3 – Opens an interactive rebase session for the last 3 commits, allowing you to squash, edit, or reorder them into a cleaner single commit history.
Viewing Rebased History & Commit Details: git log --oneline – Displays a compact history showing the newly squashed commit structure on the active feature branch.
git show <commit-hash> – Inspects the metadata, commit message, and full file diff contents of a specific commit (e.g., showing the combined code changes in feature.txt).

```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$ echo "step 1 code"> feature.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$ git add feature.txt
warning: in the working copy of 'feature.txt', LF will be replaced by CRLF the next
ime Git touches it

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$ git commit -m "doing"
[feature 0ecc68e] doing
 1 file changed, 1 insertion(+)
 create mode 100644 feature.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$ echo "step 2 code" >> feature.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$ git commit -am "fixing"
warning: in the working copy of 'feature.txt', LF will be replaced by CRLF the next
ime Git touches it
[feature 89cd9b6] fixing
 1 file changed, 1 insertion(+)

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$ echo "step 3 code" >> feature.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$ git commit -am "final touchups"
warning: in the working copy of 'feature.txt', LF will be replaced by CRLF the next
ime Git touches it
[feature 9c28713] final touchups
 1 file changed, 1 insertion(+)

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (feature)
$ git rebase -i HEAD~3
[detached HEAD fef4321] doing
 Date: Mon Oct 26 16:29:17 2026 +0530
 1 file changed, 3 insertions(+)
 create mode 100644 feature.txt
Successfully rebased and updated refs/heads/feature.
Deletion of directory '.git/rebase-merge' failed. Should I try again? (y/n) n
error: could not remove '.git/rebase-merge'

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
```

```
 Date: Mon Oct 26 16:29:17 2026 +0530
 1 file changed, 3 insertions(+)
 create mode 100644 feature.txt
Successfully rebased and updated refs/heads/feature.
Deletion of directory '.git/rebase-merge' failed. Should I try again? (y/n) n
error: could not remove '.git/rebase-merge'

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git log --online
fatal: unrecognized argument: --online

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git log --oneline
fef4321 (HEAD -> feature) doing
fa5e523 add bad code
4030faa second attempt for conflict2
c5c7796 second attempt for conflict practice
f872dde Revert "added a new file1 in main branch"
6d721ed Ignore My secret notes file
bc9e098 add passwords.txt to gitignore
b557cec (origin/hotfix) added a new file1 in main branch
dd3c2e4 Merge pull request #1 from mdtanveer801/feature-branch
f50c0b6 (origin/feature-branch) Add sample.txt in feature branch
61eddc5 add ahmed1.txt and ahmed2.txt
3edf1a1 Initial commit

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git show
commit fef43216c3a2a07539994e4ab744ac71721134b9 (HEAD -> feature)
Author: Mohammed Tanveer Ahmed <mdahmedaws3105@gmail.com>
Date:   Mon Oct 26 16:29:17 2026 +0530

    doing

    fixing

diff --git a/feature.txt b/feature.txt
new file mode 100644
index 0000000..ed68f13
--- /dev/null
+++ b/feature.txt
@@ -0,0 +1,3 @@
+step 1 code
+step 2 code
+step 3 code

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$
```

7) Tagging & Release

Create a version tag (v1.0), push it to GitHub, then delete and restore it.

Tag Management & Remote Synchronization

Creating and Pushing Tags:

git tag <tag-name> – Creates a local lightweight tag marking a release version (e.g., git tag v1.0).

git push origin <tag-name> – Pushes the specified tag up to the remote GitHub repository.

Deleting Local and Remote Tags:

git tag -d <tag-name> – Deletes a tag locally from your repository. git push origin --delete <tag-name> – Deletes a tag from the remote repository.

Directory Navigation & Environment Checking:cd .. – Moves up to the parent directory.

ll – Lists directory contents with detailed file permissions.

cd <directory> – Navigates into the working project folder.

```
$ git tag v1.0

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ cd..
bash: cd..: command not found

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ cd ..

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop
$ ll
total 10097
-rwxr-xr-x 1 mdtan 197609    2299 May 21 13:37 'Amazon Chime.lnk'*
-rw-r--r-- 1 mdtan 197609   56022 Oct 21 14:25 'Appointment Confirmation  · Customer
Self-Service.pdf'
-rw-r--r-- 1 mdtan 197609  406416 Oct 19 16:43 'Bash Scriptiing 1.pdf'
-rw-r--r-- 1 mdtan 197609 1073635 Oct 17 15:21 'Challange on jenkins docker tomcat.do
cx'
-rw-r--r-- 1 mdtan 197609 1880883 Oct 17 17:19  Challenges-1.pdf
-rwxr-xr-x 1 mdtan 197609    2320 Oct 13 17:03 'Docker Desktop.lnk'*
-rw-r--r-- 1 mdtan 197609 1626264 Oct 24 20:04 'Git-GIThub Task.pdf'
drwxr-xr-x 1 mdtan 197609       0 Oct 26 16:35  Git_practice2/
-rwxr-xr-x 1 mdtan 197609    1318 Sep 10 18:22 'Microsoft Edge.lnk'*
-rwxr-xr-x 1 mdtan 197609    2400 Aug 31  2024 'Mohammed Tanveer - Chrome.lnk'*
-rwxr-xr-x 1 mdtan 197609    2454 May  8 17:57 'Personal - Edge.lnk'*
-rwxr-xr-x 1 mdtan 197609    1522 Oct 25 22:16  Slack.lnk*
-rw-r--r-- 1 mdtan 197609 5237519 Oct 25 19:40 'Task 1.docx'
-rwxr-xr-x 1 mdtan 197609    1332 Sep  9 14:46 'Typing Master 12.lnk'*
-rwxr-xr-x 1 mdtan 197609    2128 Sep 15 18:44  Webex.lnk*
-rw-r--r-- 1 mdtan 197609     474 Oct 22 09:48  desktop.ini
drwxr-xr-x 1 mdtan 197609       0 Oct 26 12:34  practice/

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop
$ cd Git_practice2

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git branch main
fatal: a branch named 'main' already exists

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git push origin v1.0
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 562 bytes | 281.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To github.com:mdtanveer801/Git_practice2.git
 * [new tag]         v1.0 -> v1.0

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git tag -d v1.0
```

```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git push origin v1.0
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 562 bytes | 281.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To github.com:mdtanveer801/Git_practice2.git
 * [new tag]         v1.0 -> v1.0

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git tag -d v1.0
Deleted tag 'v1.0' (was fef4321)

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git push origin --delete v1.0
To github.com:mdtanveer801/Git_practice2.git
 - [deleted]         v1.0

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git tag v1.0

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$ git push origin v1.0
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 852 bytes | 426.00 KiB/s, done.
Total 9 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To github.com:mdtanveer801/Git_practice2.git
 * [new tag]         v1.0 -> v1.0

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|REBASE)
$
```

8)Clone with Sparse Checkout

Clone only a subdirectory of a repo using sparse checkout.

Definition: What does "Sparse" mean?

In Git, "sparse" means selective, thin, or scattered rather than complete or dense. A sparse checkout is a feature that allows you to work with massive repositories by instructing Git to check out and display only a restricted subset of files or directories, leaving the rest of the repository hidden and saving local disk space.

Sparse Checkout Reference & Workflow1.

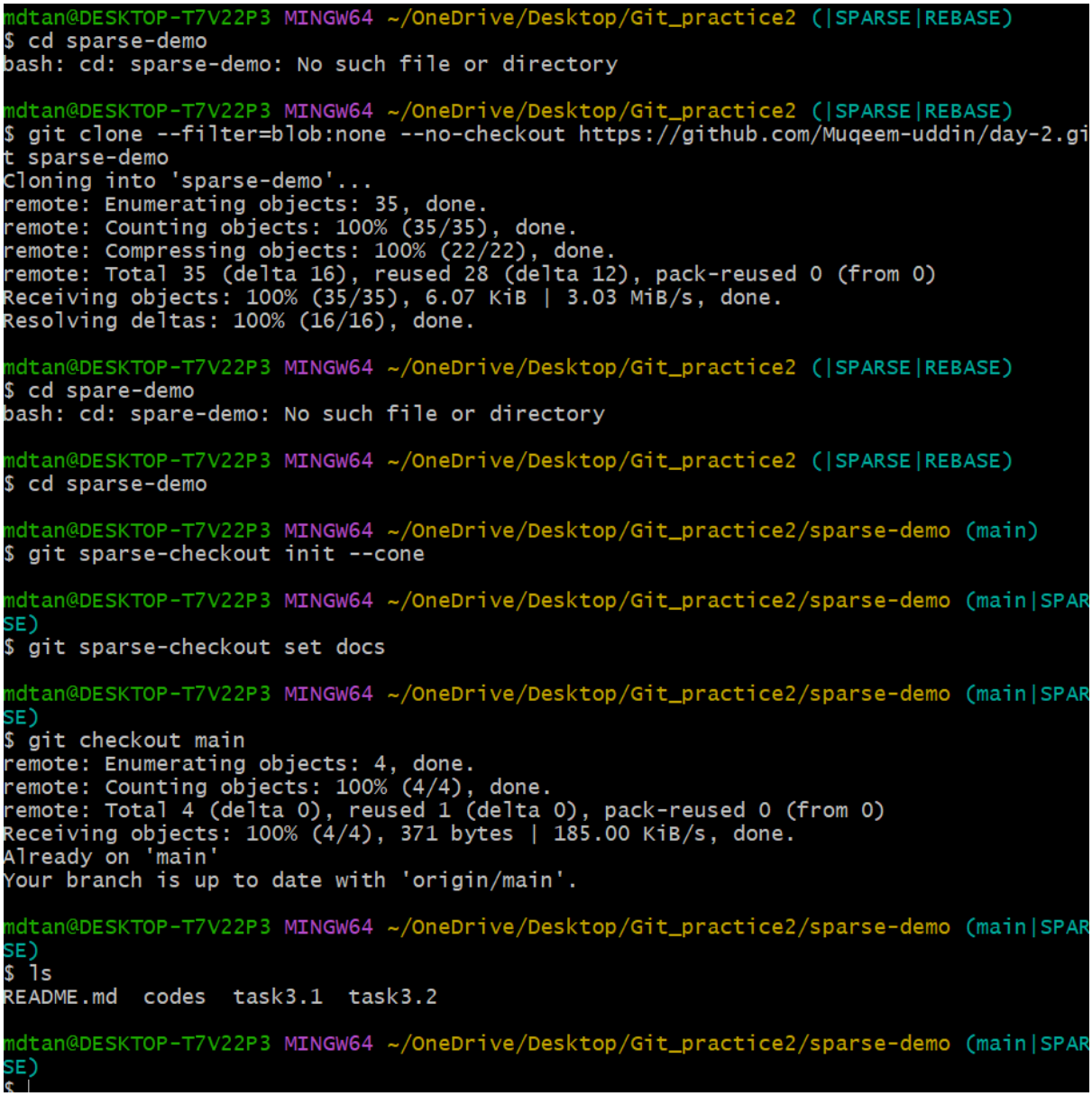
Clone Without Populating Files:

git clone --no-checkout <repository-url> : Clones the repository metadata and history without populating the working directory with files right away.

2. Initialize Sparse Checkout:

git sparse-checkout init --cone

Use: Enables sparse-checkout mode in a optimized directory "cone" pattern. 3. Set Target Files or Directories:
git sparse-checkout set <file-or-folder-path> Use: Explicitly defines which specific files or folders (e.g., abcd.docx) you want populated in your workspace while ignoring the rest of the repository.
4. Verify Workspace:
ls Use: Lists the contents of your directory to confirm the sparse checkout filter has been successfully applied.



Step-by-Step Explanation (Roman English)

1. **Repository Clone Karna (with filters):**
    - git clone --filter=blob:none --no-checkout https://github.com/Muqeem-uddin/day-2.git sparse-demo
    - *Matlab:* Yahan aapne --filter=blob:none aur --no-checkout ke sath ek repository clone ki hai aur folder ka naam sparse-demo rakha hai. Iska matlab hai ke file contents (blobs) shuru mein download nahi honge jab tak aap unhe demand na karein.
2. **Folder Ke Andar Jana:**

    - cd sparse-demo
    - *Matlab:* Aap newly cloned folder ke andar enter ho gaye hain.
3. **Sparse Checkout Init Karna:**
    - git sparse-checkout init --cone
    - *Matlab:* Aapne sparse-checkout mode ko --cone flag ke sath initialize kar diya hai.
4. **Specific Folder Set Karna:**
    - git sparse-checkout set docs
    - *Matlab:* Aapne Git ko bataya ke mujhe sirf docs naam ka folder active karna hai.
5. **Checkout & Verify Karna:**
    - git checkout main
    - ls
    - *Matlab:* Branch checkout karne ke baad jab aapne ls command chalayi, toh aap dekh sakte hain ke working directory mein sirf wahi files/folders nazar aa rahe hain jo sparse checkout ke rules ke mutabiq set kiye thay (README.md, codes, task3.1, task3.2).

Documentation Notes (For Your Project)

- **Cloning with Blobs Filter & No Checkout:**

Bash
git clone --filter=blob:none --no-checkout <repo-url> <folder-name>

    - *Use:* Clones repository structure and history efficiently without downloading heavy file contents upfront.

- **Initializing and Setting Path:**

Bash
git sparse-checkout init --cone
git sparse-checkout set <folder-name>

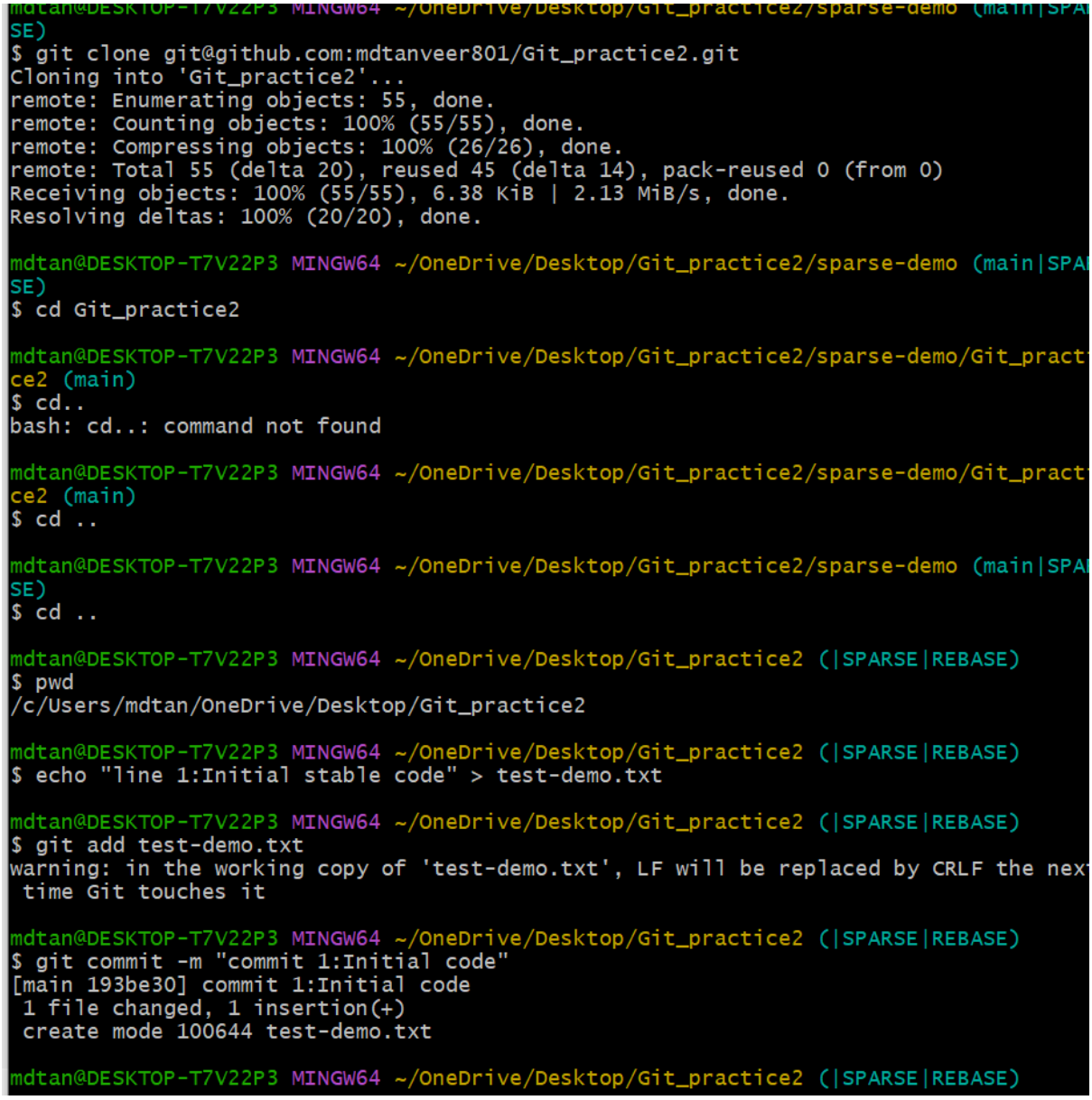
    - *Use:* Enables sparse checkout mode and isolates only the required folder or directory in your workspace.

9)Reset vs Revert Challenge
Demonstrate the difference between git reset --hard and git revert in a repo.

Navigation & Initial Setup
git clone ... / cd .. / pwd
Use: Navigated through directories and verified your active working path (/C/Users/mdtan/OneDrive/Desktop/Git_practice2).
2. Creating Initial Stable Code
echo "line 1:Initial stable code" > test-demo.txt
Use: Creates a new text file test-demo.txt containing the first line.git add test-demo.txt
Use: Stages the file for the initial commit.git commit -m "commit 1:Initial code"
Use: Commits the stable code to the repository with a clear message.
3. Introducing and Reverting a Bugecho "line 2: Buggy code introduced" >> test-demo.txt Use: Appends a second line containing a bug to the file.git commit -am "commit 2: added a bug" Use: Automatically stages (-a) and commits (-m) the bug update.
git revert HEAD --no-edit Use: Safely undoes the latest commit (HEAD) by creating a new inverse commit that removes the bug, without opening a text editor prompt.
4. Experimental Work & Hard Reset

echo "Line 3: Another experimental change" >> test-demo.txt Use: Appends a third experimental line to the file.git commit -am "Commit
3: Experimental work"
Use: Commits the experimental changes. git reset --hard HEAD~2
Use: Forcefully rolls back the repository state by 2 commits, discarding everything up to commit 2: added a bug and resetting the working tree.

6. Verification & History Inspection
7. git log --oneline Use: Displays a compact history of all commits to confirm the current position of the HEAD pointer. cat test-demo.txt
8. Use: Prints the contents of test-demo.txt to the terminal to verify that the file contents match the expected state after the reset.



```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ echo "line 2: Buggy code introduced" >> test-demo.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git commit -am "commit 2: added a bug"
warning: in the working copy of 'test-demo.txt', LF will be replaced by CRLF the next
 time Git touches it
[main 7c236cb] commit 2: added a bug
 1 file changed, 1 insertion(+)

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git revert HEAD --no-edit
[main a42eb5e] Revert "commit 2: added a bug"
 Date: Mon Oct 26 19:29:43 2026 +0530
 1 file changed, 1 deletion(-)

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ echo "Line 3: Another experimental change" >> test-demo.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git commit -am "Commit 3: Experimental work"
warning: in the working copy of 'test-demo.txt', LF will be replaced by CRLF the next
 time Git touches it
[main 99f0b53] Commit 3: Experimental work
 1 file changed, 1 insertion(+)

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ # Find your safe commit hash using git log, or use HEAD~2 to go back 2 commits

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git reset --hard HEAD~2
HEAD is now at 7c236cb commit 2: added a bug

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git log --oneline
7c236cb (HEAD -> main) commit 2: added a bug
193be30 commit 1:Initial code
a54b935 (tag: v1.0, release) two files are combines successfully in one commit.
9587a7d my forget code added
07bd91c my first code added
e744edd (origin/main, origin/HEAD) Revert "add bad code"
eb2a0d2 add bad code
c4b2ac3 second attempt for 2
4030faa second attempt for conflict2
c5c7796 second attempt for conflict practice
f872dde Revert "added a new file1 in main branch"
```

```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git log --oneline
7c236cb (HEAD -> main) commit 2: added a bug
193be30 commit 1:Initial code
a54b935 (tag: v1.0, release) two files are combines successfully in one commit.
9587a7d my forget code added
07bd91c my first code added
e744edd (origin/main, origin/HEAD) Revert "add bad code"
eb2a0d2 add bad code
c4b2ac3 second attempt for 2
4030faa second attempt for conflict2
c5c7796 second attempt for conflict practice
f872dde Revert "added a new file1 in main branch"
6d721ed Ignore My secret notes file
bc9e098 add passwords.txt to gitignore
b557cec (origin/hotfix) added a new file1 in main branch
dd3c2e4 Merge pull request #1 from mdtanveer801/feature-branch
f50c0b6 (origin/feature-branch) Add sample.txt in feature branch
61eddc5 add ahmed1.txt and ahmed2.txt
3edf1a1 Initial commit

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ cat test-demo.txt
line 1:Initial stable code
line 2: Buggy code introduced

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
```

10) Detached HEAD Challenge
Check out a specific commit (detached HEAD state) and create a new branch from it.

Normally, when working in Git, you are attached to a branch (like main), and your commits move that branch forward automatically.

A "Detached HEAD" state occurs when you check out a specific commit hash instead of a branch name, stepping backward in time to inspect or test past code.

The Risk: Commits made while detached are not tied to any branch, meaning they can easily be lost when you switch away.

The Solution: To save any work or continue coding from that historical point, you instantly create a brand-new branch right from it!

Step-by-Step Walkthrough
Let's practice entering a detached HEAD state and capturing it into a new branch inside your Git_practice2 repository:

Step 1: Create a couple of commits for your history
If your repository is empty or you want fresh history to test with, run:

Bash
```
echo "Version 1" > test-history.txt
git add test-history.txt
git commit -m "Commit A: First version"

echo "Version 2" >> test-history.txt
git commit -am "Commit B: Second version"
```
Step 2: Grab the commit hash of Commit A
Run your log to find the hash for "Commit A":

Bash
```
git log --oneline
```
You will see output similar to this:

Plaintext
```
b2c3d4e Commit B: Second version
a1b2c3d Commit A: First version  <-- (Copy this hash)
```
Copy the short hash for Commit A (e.g., a1b2c3d).

Step 3: Checkout the commit (Enter Detached HEAD)
Check out that specific commit directly:

Bash
```
git checkout a1b2c3d
```

What Git tells you: Git will output a warning message stating you are in a "detached HEAD" state, meaning you are looking directly at a past snapshot of code rather than a living branch. If you inspect test-history.txt, you will see it only contains "Version 1".

Step 4: Create a new branch from this detached state
To save your place and escape the detached state, create a permanent branch right from here:

Bash
git checkout -b saved-investigation-branch
You are now safely on your new branch (saved-investigation-branch), completely out of the detached HEAD state, and free to commit normally!

```
ndtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ echo "Version 1" > test-history.txt

echo "Version 2" >> test-history.txt
git commit -am "Commit B: Second version"
ndtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git add test-history.txt
warning: in the working copy of 'test-history.txt', LF will be replaced by CRLF the n

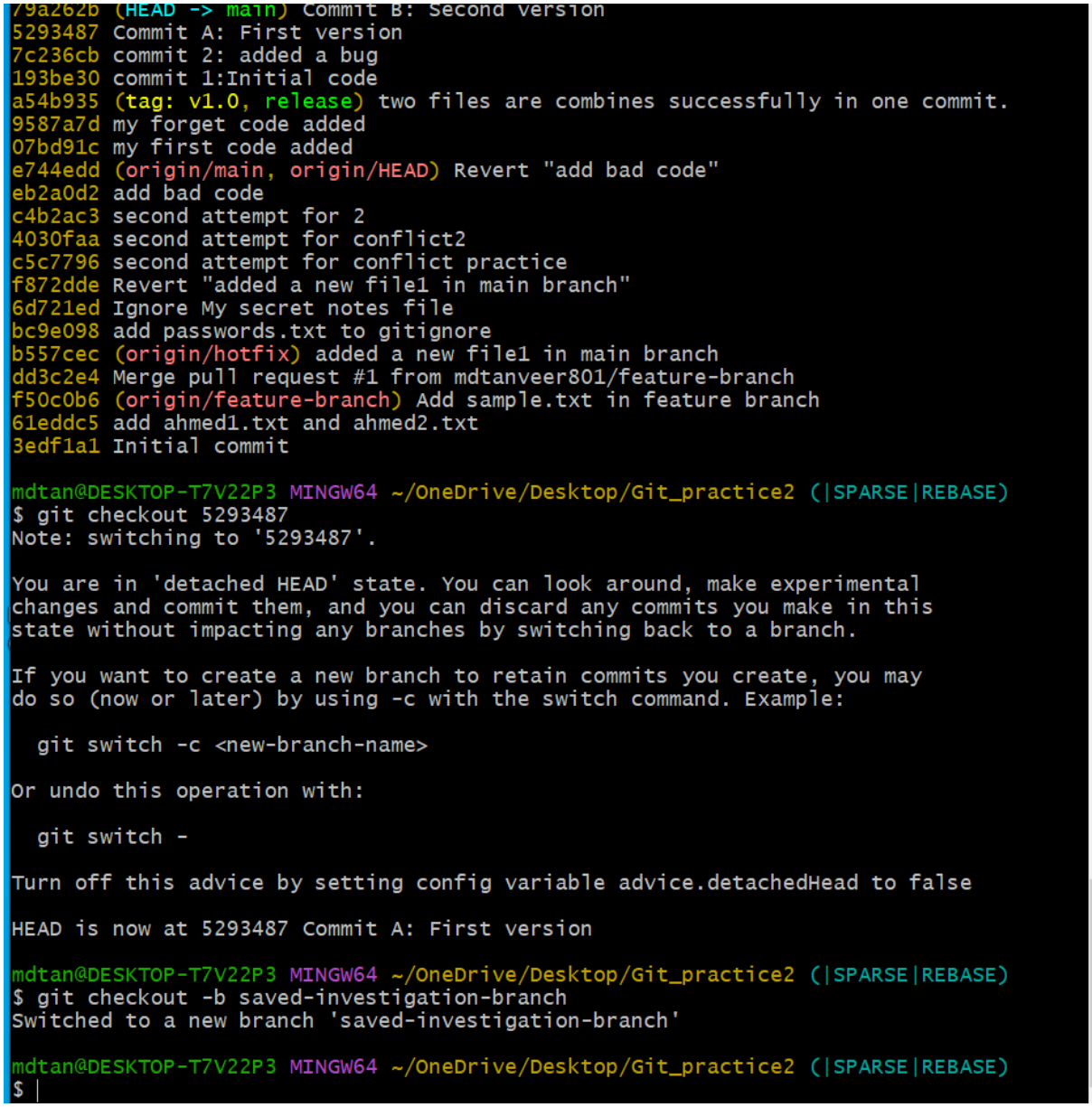
ndtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git commit -m "Commit A: First version"
[main 5293487] Commit A: First version
 1 file changed, 1 insertion(+)
 create mode 100644 test-history.txt

ndtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$

ndtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ echo "Version 2" >> test-history.txt

ndtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git commit -am "Commit B: Second version"
warning: in the working copy of 'test-history.txt', LF will be replaced by CRLF the n
ext time Git touches it
[main 79a262b] Commit B: Second version
 1 file changed, 1 insertion(+)

ndtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git log --oneline
79a262b (HEAD -> main) Commit B: Second version
5293487 Commit A: First version
fc236cb commit 2: added a bug
```



11) **Git Hooks Challenge**

Configure a pre-commit hook to reject commits without a message format (e.g., must start with JIRA-XXX)

: Open and edit the hook file

Run this command in your terminal to open the commit-msg file (or you can open it in VS Code):

Bash

nano .git/hooks/commit-msg

(If nano opens, paste the validation script code inside it. If you are using Notepad or VS Code, just open .git/hooks/commit-msg, paste the script below, and save it).

Here is the exact script to paste inside:

Bash

#!/bin/sh

commit_msg_file=$1

commit_msg=$(cat "$commit_msg_file")

```
if ! echo "$commit_msg" | grep -qE "^JIRA-[0-9]+"; then
  echo "-------------------------------------------------"
  echo "❌ ERROR: Commit rejected!"
  echo "Your commit message must start with a JIRA ID (e.g., JIRA-123)."
  echo "-------------------------------------------------"
  exit 1
fi
exit 0
```

(If using nano, press Ctrl + O then Enter to save, and Ctrl + X to exit).

Step 2: Make it executable again

Back in your terminal, run this to make sure Git has permission to run it:

Bash

```
chmod +x .git/hooks/commit-msg
```

Step 3: Test your commit again!

Now try your test commits from your project root folder:

Bash

```
echo "test changes" > sample.txt
git add sample.txt

# This should FAIL and get blocked:
git commit -m "fixed a bug without jira tag"

# This should SUCCEED:
git commit -m "JIRA-404: fixed a bug properly"
```

```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ nano .git/hooks/commit-msg

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ chmod +x .git/hooks/commit-msg

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ echo "test changes" > sample.txt
# This should FAIL and get blocked:
git commit -m "fixed a bug without jira tag"

# This should SUCCEED:
git commit -m "JIRA-404: fixed a bug properly"
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git add sample.txt
warning: in the working copy of 'sample.txt', LF will be replaced by CRLF the nex
me Git touches it

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ # This should FAIL and get blocked:

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git commit -m "fixed a bug without jira tag"
--------------------------------------------------
x  ERROR: Commit rejected!
Your commit message must start with a JIRA ID (e.g., JIRA-123).
--------------------------------------------------

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ # This should SUCCEED:

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git commit -m "JIRA-404: fixed a bug properly"
[saved-investigation-branch bf5c581] JIRA-404: fixed a bug properly
 1 file changed, 1 insertion(+), 1 deletion(-)

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$
```

12) Squash Merge vs Rebase Merge

Show the difference between squash merge and rebase merge with evidence.

Branch (`git checkout -b`): Think of a branch as your own private workspace copy of the project. It lets you test or build new things without breaking the main working code.

Commit (`git commit`):A permanent snapshot or save point of your code at a specific moment in time.

Squash (`git merge --squash`): A technique where multiple small, messy commit saves from a feature branch are combined ("squashed") into one single clean commit** before merging into the main branch.

Git Hook / Validation Rule: A rule enforced by your repository (in your case, requiring a JIRA ID like `JIRA-123`) that rejects your commit if the message doesn't follow the proper format.

Step-by-Step Breakdown of What Happened

1. Creating the Feature Branch & Making Changes

Command: `git checkout -b feature-merge-test`

What it does: Created and switched you over to a new branch called `feature-merge-test`.

Commands`echo "Feature part 1" > feature.txt` & `echo "Feature part 2" >> feature.txt`

What it does:Created a file named `feature.txt` and added two lines of text to it over time.
2. The Commit Rejections

Commands: `git commit -m ...` and `git commit -am ...`
What happened: Every time you tried to save your changes with a commit, Git rejected it** with an error: `Your commit message must start with a JIRA ID (e.g., JIRA-123)`. Your project's policy requires a ticket number in the message.
3. Switching Back and Squashing
Command:`git checkout development`
What it does:Switched you back to your `development` branch.
Command: `git merge --squash feature-merge-test`
What it does: Grabbed all the work from your `feature-merge-test` branch and prepared it to be merged as one combined set of changes.
Command:`git commit -m "Squashed: Complete feature implementation"`
What happened:You tried to save the squashed changes, but it got **rejected again** because it was still missing the required JIRA ID prefix.
4. Checking History
Command: `git log`
What it does: Prints out a chronological list of all previous successful saves (commits) on your current branch so you can review who made changes and when.

```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git checkout -b feature-merge-test
git commit -m "Add part 1 of feature"

echo "Feature part 2" >> feature.txt
git commit -am "Add part 2 of feature"Switched to a new branch 'feature-merge-test'

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ echo "Feature part 1" > feature.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git add feature.txt
warning: in the working copy of 'feature.txt', LF will be replaced by CRLF the next t
ime Git touches it

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git commit -m "Add part 1 of feature"
--------------------------------------------------
✗ ERROR: Commit rejected!
Your commit message must start with a JIRA ID (e.g., JIRA-123).
--------------------------------------------------

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ echo "Feature part 2" >> feature.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git commit -am "Add part 2 of feature"
warning: in the working copy of 'feature.txt', LF will be replaced by CRLF the next t
ime Git touches it
--------------------------------------------------
✗ ERROR: Commit rejected!
Your commit message must start with a JIRA ID (e.g., JIRA-123).
--------------------------------------------------

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git checkout development
Switched to branch 'development'
A       feature.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git merge --squash feature-merge-test
```

```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git merge --squash feature-merge-test
Your local changes to the following files would be overwritten by merge:
  feature.txt
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git commit -m "Squashed: Complete feature implementation"
--------------------------------------------------
✗ ERROR: Commit rejected!
Your commit message must start with a JIRA ID (e.g., JIRA-123).
--------------------------------------------------

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2 (|SPARSE|REBASE)
$ git log
commit 1529de57b7506123efdac1c9dc3be96f1b54d496 (HEAD -> development, origin/development
)
Merge: d8150fa 2303866
Author: Mohammed Tanveer Ahmed <mdahmedaws3105@gmail.com>
Date:   Mon Oct 26 15:19:02 2026 +0530

    done

commit d8150fac73150c8b14e379caa93b325c3c84a70d
Author: Mohammed Tanveer Ahmed <mdahmedaws3105@gmail.com>
Date:   Mon Oct 26 15:17:51 2026 +0530

    added develp file

commit 23038669a0b65ca1194f0207761d1a743c814830 (hotfix)
Author: Mohammed Tanveer Ahmed <mdahmedaws3105@gmail.com>
Date:   Mon Oct 26 15:15:40 2026 +0530

    added a file

commit c5c7796915d798afb88671230df2ca8e057e66c3
Author: Mohammed Tanveer Ahmed <mdahmedaws3105@gmail.com>
Date:   Mon Oct 26 14:55:51 2026 +0530

    second attempt for conflict practice

commit f872dde66f7d8746add4e2adc893614c0aee1176
Author: Mohammed Tanveer Ahmed <mdahmedaws3105@gmail.com>
Date:   Sat Oct 24 18:15:34 2026 +0530

    Revert "added a new file1 in main branch"
```

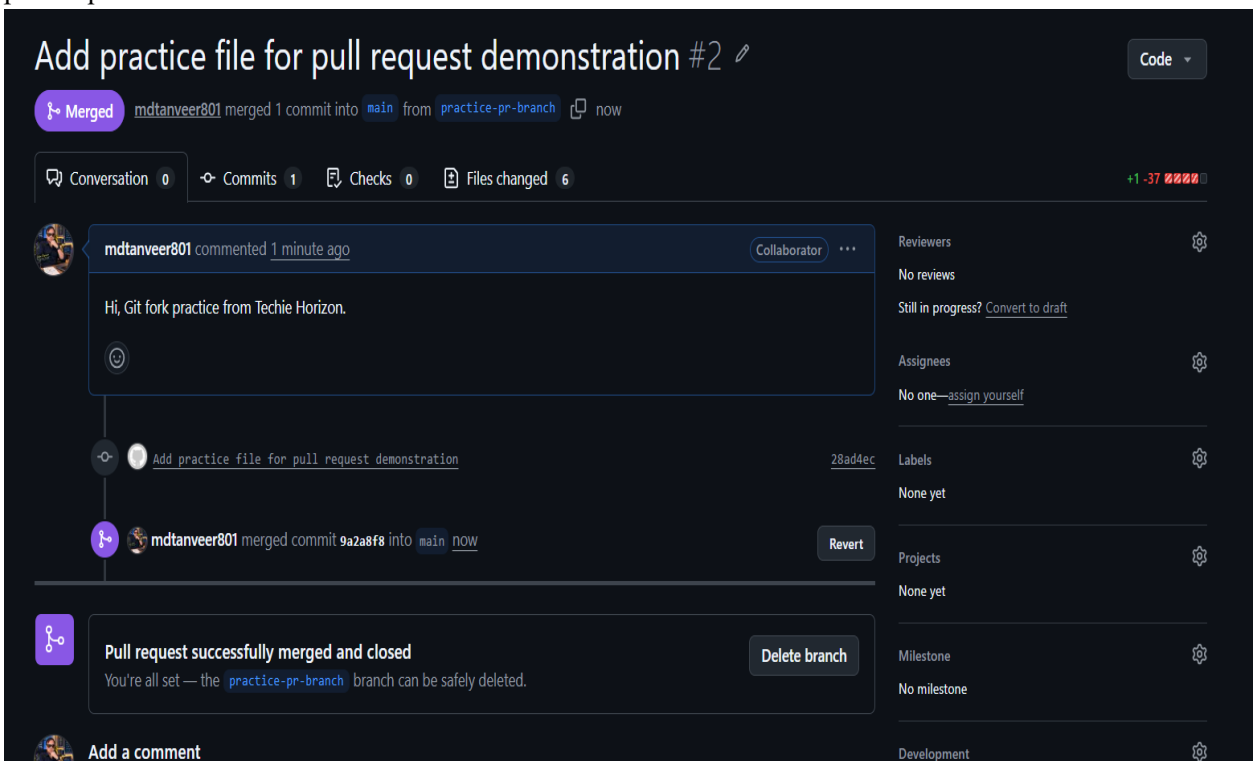
13)Fork & Pull Request Workflow
Fork a repo, make a change, and submit a pull request to the original repo.

Run these commands in your terminal:
Bash
1. Move directly into your already-cloned day-2 folder
cd day-2

2. Verify you are inside it
git status
Once you are inside the day-2 folder, you can proceed with creating your branch, making your change, and pushing it to your fork:
Bash
3. Create and switch to your practice branch
git checkout -b practice-pr-branch

4. Make a test file
echo "Practicing pull request workflow" > pr-test.txt
git add pr-test.txt
git commit -m "Add practice file for pull request demonstration"

5. Push your branch to your fork on GitHub
git push -u origin practice-pr-branch
After running git push, head over to GitHub in your web browser, go to your fork, click "Compare & pull request"



```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (main)
$ # 3. Create and switch to your practice branch
# 4. Make a test file
echo "Practicing pull request workflow" > pr-test.txt
git add pr-test.txt
git commit -m "Add practice file for pull request demonstration"

# 5. Push your branch to your fork on GitHub
git push -u origin practice-pr-branch
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (main)
$ git checkout -b practice-pr-branch
Switched to a new branch 'practice-pr-branch'

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch
)
$

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch
)
$ # 4. Make a test file

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch
)
$ echo "Practicing pull request workflow" > pr-test.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch
)
$ git add pr-test.txt
warning: in the working copy of 'pr-test.txt', LF will be replaced by CRLF the next time
 Git touches it

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch
)
$ git commit -m "Add practice file for pull request demonstration"
[practice-pr-branch remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 1 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), 371 bytes | 371.00 KiB/s, done.
28ad4ec] Add practice file for pull request demonstration
 6 files changed, 1 insertion(+), 37 deletions(-)
 delete mode 100644 .gitignore
 delete mode 100644 README.md
 delete mode 100644 codes
 create mode 100644 pr-test.txt
 delete mode 100644 task3.1
 delete mode 100644 task3.2
```

```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch
)
$ git commit -m "Add practice file for pull request demonstration"
[practice-pr-branch remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 1 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), 371 bytes | 371.00 KiB/s, done.
28ad4ec] Add practice file for pull request demonstration
 6 files changed, 1 insertion(+), 37 deletions(-)
 delete mode 100644 .gitignore
 delete mode 100644 README.md
 delete mode 100644 codes
 create mode 100644 pr-test.txt
 delete mode 100644 task3.1
 delete mode 100644 task3.2

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch
)
$

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch
)
$ # 5. Push your branch to your fork on GitHub

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch
)
$ git push -u origin practice-pr-branch
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (1/1), done.
Writing objects: 100% (3/3), 319 bytes | 319.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'practice-pr-branch' on GitHub by visiting:
remote:      https://github.com/Muqeem-uddin/day-2/pull/new/practice-pr-branch
remote:
To https://github.com/Muqeem-uddin/day-2.git
 * [new branch]      practice-pr-branch -> practice-pr-branch
branch 'practice-pr-branch' set up to track 'origin/practice-pr-branch'.

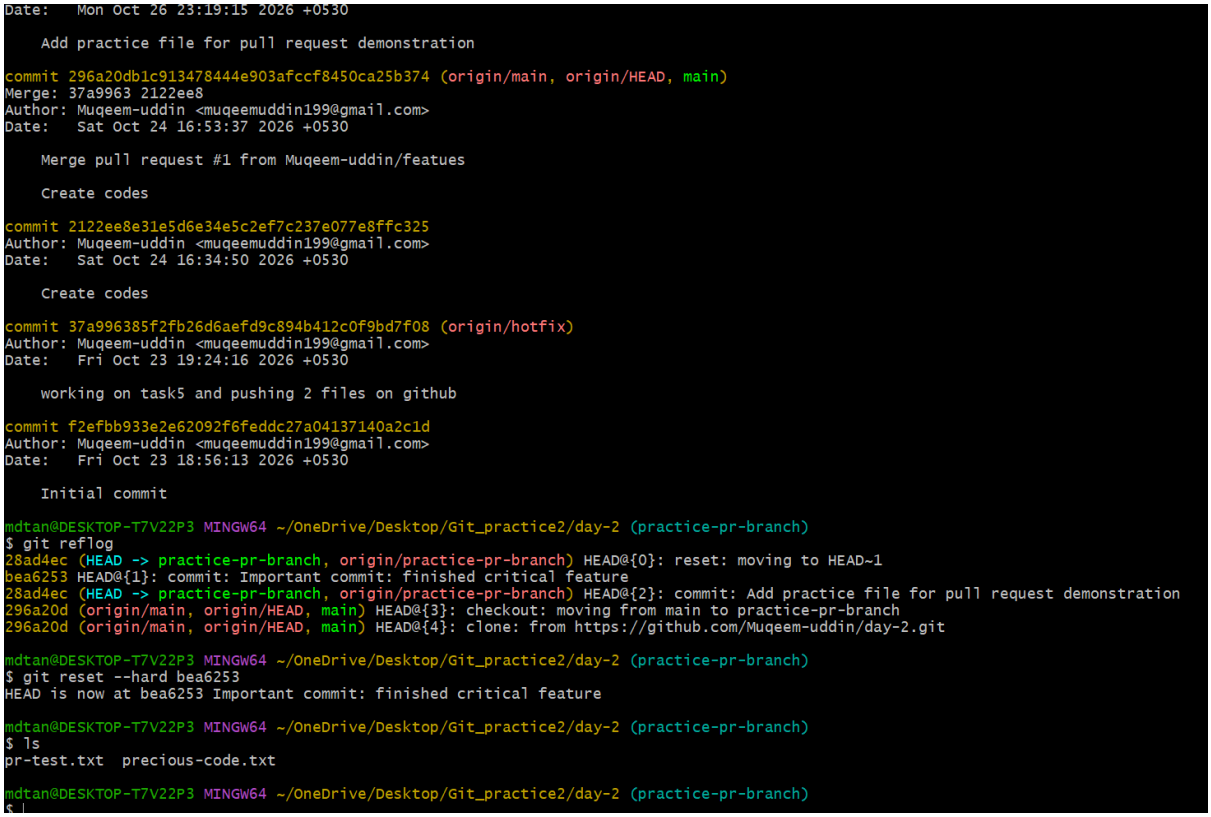
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch
)
```

14) Recover Lost Commit

Commit something, reset hard, and then recover it using git reflog

In this practice session, you simulated a common panic scenario—accidentally deleting/losing a commit using a hard reset, and then successfully recovering it using Git's safety net (reflog). Step-by-Step Breakdown & Definitions

1. Creating and Saving a Critical Fileecho "This is important work..." > precious-code.txt What it does: Creates a new file named precious-code.txt and writes text inside it.git add precious-code.txt
2. What it does: Stages the file so it is ready to be saved.git commit -m "Important commit: finished critical feature"
3. What it does: Permanently saves the snapshot to your repository (creating commit hash bea6253).
4. 2. The Dangerous Move (Rolling Back) git reset --hard HEAD~1
5. What it does: Hard Reset. This command rolls your project back by 1 commit (HEAD~1), completely wiping out your recent work and returning you to the previous state (28ad4ec).
6. It makes it look like your precious file and commit are gone forever! 3. The Rescue Mission (Using Reflog)git reflog
7. What it does: Reference Log. Think of this as Git's black box flight recorder. It tracks every single movement of your pointer (HEAD).
8. Even when you "lose" a commit, reflog remembers its ID (bea6253). git reset --hard bea6253
9. What it does: Uses the hash found in the reflog to jump straight back to your "lost" commit, instantly bringing your code and files back to life.
10. ls What it does: Lists the files in your current folder to verify that your recovered file (precious-code.txt) has safely returned.



```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch)
$ echo "This is important work I cannot afford to lose" > precious-code.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch)
$ git add precious-code.txt
warning: in the working copy of 'precious-code.txt', LF will be replaced by CRLF the next time Git touches it

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch)
$ git commit -m "Important commit: finished critical feature"
[practice-pr-branch bea6253] Important commit: finished critical feature
 1 file changed, 1 insertion(+)
 create mode 100644 precious-code.txt

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch)
$ git reset --hard HEAD~1
HEAD is now at 28ad4ec Add practice file for pull request demonstration

mdtan@DESKTOP-T7V22P3 MINGW64 ~/OneDrive/Desktop/Git_practice2/day-2 (practice-pr-branch)
$ git log
commit 28ad4ec40f4c9b87a625c003bfa462d4e317f5f7 (HEAD -> practice-pr-branch, origin/practice-pr-branch)
Author: Mohammed Tanveer Ahmed <mdahmedaws3105@gmail.com>
Date:   Mon Oct 26 23:19:15 2026 +0530

    Add practice file for pull request demonstration

commit 296a20db1c913478444e903afccf8450ca25b374 (origin/main, origin/HEAD, main)
Merge: 37a9963 2122ee8
Author: Muqeem-uddin <muqeemuddin199@gmail.com>
Date:   Sat Oct 24 16:53:37 2026 +0530

    Merge pull request #1 from Muqeem-uddin/featues

    Create codes

commit 2122ee8e31e5d6e34e5c2ef7c237e077e8ffc325
Author: Muqeem-uddin <muqeemuddin199@gmail.com>
Date:   Sat Oct 24 16:34:50 2026 +0530

    Create codes

commit 37a996385f2fb26d6aefd9c894b412c0f9bd7f08 (origin/hotfix)
Author: Muqeem-uddin <muqeemuddin199@gmail.com>
Date:   Fri Oct 23 19:24:16 2026 +0530

    working on task5 and pushing 2 files on github

commit f2efbb933e2e62092f6feddc27a04137140a2c1d
Author: Muqeem-uddin <muqeemuddin199@gmail.com>
```